

The Intermediate Coding Agents Guide

Mudit Jain / @ai_with_mudit

September 8, 2026

CONTENTS

The Intermediate Coding Agents Guide	
Subagents, permanent memory, and using an agent as a second brain	
The one idea underneath all three	
Part 1 — Deploying subagents	
The empty head	
What a brief has to contain	
Write the ones you reuse down	
Fanning out	
Split by dimension, not by folder	
Where the files come in	
What actually breaks at scale	
Part 2 — Permanent memory	
Memory is a file that gets read at startup	
What goes in it	
Length is the whole game	
The Claude Code catch	
The hierarchy	
Memory that maintains itself	
The only test that counts	
Part 3 — A second brain	
First, use what is already there	

The difference from Part 2

Retrieval, not volume

One fact per file

The index is the brain

Let the agent write it

Testing it honestly

Keep it plain

Part 4 — Making it hold

Skills — for what you repeat

Hooks — for what must not happen

Where to start

Do it instead of reading it

The Intermediate Coding Agents Guide

SUBAGENTS, PERMANENT MEMORY, AND USING AN AGENT AS A SECOND BRAIN

This guide is for people who already use a coding agent and have hit a ceiling.

You can open Claude Code or Codex, describe a change, and get one. You have a feel for what it is good at. And you have started to notice that the gains stopped — you are still working one prompt at a time, still explaining the same things every session, and the agent still forgets everything the moment you close the window.

That ceiling is not the model's. It is an information problem, and it has three fixes. This guide is those three.

Verified on 8 September 2026, Windows 11, Claude Code 2.1.202 — with every claim about hooks, memory, subagents and skills checked against the official documentation on that date rather than written from memory. Behaviour changes between versions: if what you see disagrees with this guide, trust what you see.

The one idea underneath all three

An agent does not remember. It reads.

Every session, every subagent, every parallel run starts from nothing and reconstructs its understanding from two sources: the prompt you gave it, and the files it can open. That is the entire mechanism.

Once that lands, the three problems in this guide stop looking like three problems:

What it feels like	What it actually is
"It forgets everything between sessions"	Nothing wrote the facts to a file it reads at startup
"Parallel agents give inconsistent results"	Each one was briefed differently, or not at all
"It keeps making the mistake I corrected last week"	The correction went into a conversation, which was thrown away

All three are the same bug. The information the agent needed was in your head or in a dead chat log, not in a file.

So the work is not better prompting. **The work is making the agent's inputs good.** Prompting runs once. A file runs forever.

Part 1 — Deploying subagents

The empty head

A subagent does not inherit your conversation. It gets the prompt you hand it and whatever it reads off the disk. Nothing else.

People meet this as a bug. They fan out five agents, the answers come back inconsistent, and they conclude parallel agents are not ready. The agents were fine. Four of them were never told what “done” meant.

It is a feature, and it is the reason the whole thing works: if every subagent carried the full conversation, you would exhaust the context window on the third one. Empty heads are what let you run twenty.

The cost is that **twenty agents is twenty briefings**, and you only get to write them once — with one agent you correct it in the next message, with eight running at once there is no next message. They all finish and hand you eight things.

What a brief has to contain

Four things. Every subagent prompt should answer all four:

1. **What to look at** — an exact path, glob, or file list. Not “the codebase”.
2. **What to produce** — the shape of the output, not the vibe of it.
3. **What not to do** — the scope limit. “Report only, do not fix.”
4. **What done looks like** — so it stops.

Most people’s first attempt has 1 and 3. The failures come from missing 2 and 4: without a fixed output shape you cannot merge the results, and without a stopping condition an agent will keep finding things until it runs out of room.

Write the ones you reuse down

A subagent you will use twice belongs in a file.

Claude Code — `.claude/agents/<name>.md` in a project, or `~/.claude/agents/` for all of them:

```
---
name: test-auditor
description: Finds tests that assert nothing useful. Use when reviewing test
quality.
tools: Read, Grep, Glob
---
```

You audit test files for tests that pass but prove nothing.

Look for:

- assertions on mocks rather than on behaviour
- tests with no assertion at all
- a test whose name claims more than its body checks

Report each as: file, line, what it claims, what it actually checks.
Report nothing else. Do not fix anything.

Codex uses `AGENTS.md` conventions and `~/.codex/skills/`. The keys differ, the discipline does not.

Two things make this file good and both are easy to skip. The `description` says *when* to use it, not just what it is — that is what the parent agent reads when choosing. And the body ends in a hard limit. Agents with no stated scope wander.

Fanning out

Launching several agents **in one message** runs them concurrently. One per message runs them in sequence, which is the same work at the same cost with none of the speedup — and it is the most common reason someone's first fan-out feels pointless.

A shape that works:

Review this repo across three dimensions, one subagent each, at the same time: (1) error handling, (2) naming and clarity, (3) dead or unreachable code. Each returns at most 5 findings as `file:line – what – why it matters`. Then merge into one list, most severe first.

Note what is doing the work there: the output format is stated **once and identically** for all three, and each has a cap. Without the shared format the merge is manual and the fan-out saved you nothing. Without the cap, one chatty agent floods the report.

Split by dimension, not by folder

This is the difference between a fan-out that helps and one that wastes money.

Six agents on six aspects of one repo gives you six different answers. **Six agents on six folders** gives you six copies of the same answer, because they are all running the same instruction.

Fan out across *questions*, not across *files*.

Where the files come in

Those three agents did not know your project's conventions. You could paste them into all three prompts. Or you could put them in a file every agent can read — `CLAUDE.md` for Claude Code, `AGENTS.md` for Codex and around thirty other tools — and brief all of them for free.

This is the hinge between Part 1 and Part 2, and if you take one line from this guide, take this one:

You scale agents by improving the files they read, not by writing longer prompts.

Twenty agents that all read one good `AGENTS.md` behave like twenty colleagues who read the onboarding doc. Twenty agents with no file behave like twenty contractors on their first morning.

What actually breaks at scale

- **There is a hard ceiling at twenty.** With twenty subagents already running, spawning another fails. `CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS` raises it, but twenty concurrent is the default wall — design a big fan-out in waves, not as one unlimited spray.

- **Going wider is cheap. Going deeper is not.** Twenty agents reading three files each is fine. Twenty reading the whole repo is slow, expensive, and returns the same finding twenty times.
 - **Cost is roughly linear.** Ten agents is ten times the tokens. Worth it when it replaces an hour; silly for something one agent does in a minute.
 - **The merge is where quality dies.** Most disappointing fan-outs are good agents plus a careless merge. Give the merge step as much thought as the prompts.
 - **Traceability matters.** Record which agent produced which finding. When one dimension is consistently wrong, you want to fix that prompt, not all of them.
-

Part 2 — Permanent memory

Memory is a file that gets read at startup

There is no hidden store. `CLAUDE.md` (Claude Code) or `AGENTS.md` (Codex, Cursor, Copilot, Gemini CLI, Aider, Zed and others) at the repo root, plain markdown, no schema, no install.

Once you see memory as “a file that is read every session”, every memory problem becomes a file problem: is the fact in it, is it findable, and is it still true?

What goes in it

Four sections will carry most projects:

```
# <project>

## What this is
One paragraph. What it does and who uses it.

## Commands
- build: <the exact command>
- test: <the exact command>
- run: <the exact command>

## Conventions
Three to five rules that are actually true here.

## Do not touch
Files or areas that need a human.
```

The commands must be pasteable. “Run the tests” is not a command; `npm test -- --run` is. This one section saves more time than the other three together, because it is the thing an agent otherwise guesses at every single session.

Length is the whole game

The file is read **every session** and competes with your actual work for the context window. Longer file, more tokens spent before anything happens, and worse adherence — the agent skims a long file the same way you do.

Keep it well under 200 lines. If it is longer, something in it belongs in a skill (loaded only when relevant) or a reference file (loaded only when pointed at).

If the agent stops following your memory file, it is too long. Halve it before you do anything else.

The Claude Code catch

This is the single most common trip-up, so it gets its own heading:

Claude Code reads `CLAUDE.md`. It does not read `AGENTS.md`.

If you wrote `AGENTS.md` for Codex or Cursor, Claude Code ignores it entirely. You do not need two copies — you need a bridge:

```
@AGENTS.md

## Claude Code
Anything that is only true for Claude goes below this line.
```

One source of truth, imported. The import path is relative to the file doing the importing.

The hierarchy

Where	Applies to	Put here
<code>~/ .claude/CLAUDE.md</code>	every project you open	how <i>you</i> like to work
<code><repo>/CLAUDE.md</code>	this project	what this project is
<code><repo>/<subdir>/CLAUDE.md</code>	loaded when Claude reads files there	rules local to that area

These do **not** override each other. They are all concatenated into context, broadest first — so there is no precedence rule to exploit, and two files that genuinely contradict each other get resolved arbitrarily. A contradiction is a bug to delete, not a puzzle to solve.

For instructions that should only apply to part of a repo, `.claude/rules/` with a `paths:` frontmatter glob is the better tool: those load only when Claude touches a matching file, so they cost no context the rest of the time.

The mistake to avoid is putting project facts in the user-level file. They follow you into every unrelated repo and quietly make the agent worse in all of them.

Memory that maintains itself

A file only you edit goes stale in a fortnight. The version that survives is the one the agent appends to:

```
## Decisions
When we decide something that is not obvious from the code – a library choice, a
pattern we rejected, a constraint from outside the repo – append it here as one
line with the date and the reason. Do not record what the code already says.
```

The last sentence is the important one. **Memory is for what is not derivable.** “This project uses React” is visible in `package.json`; writing it down costs context every session and earns nothing. “Chose Postgres over SQLite — needs concurrent writes from two services” is memory, because nothing in the repo says why.

Same rule for entries: `Chose Postgres` is useless. The reason is the note.

The only test that counts

Set it up, close the session completely, open a new one, and ask for a small change **without telling it anything about the project.**

Before you start, write down the four things you expect it to know. Then score honestly. Anything it missed is a gap, and the fix is always one of three: the fact is missing, it is buried, or it is too vague.

And prune. A wrong memory is worse than no memory, because the agent trusts it.

Part 3 — A second brain

First, use what is already there

Claude Code ships an **auto memory**: it writes its own typed notes into a per-project memory directory and keeps a `MEMORY.md` index that loads at the start of every session. If all you want is “remember my corrections”, that is built already — turn it on and skip to Part 4.

What follows is worth building anyway, for two reasons. The built-in memory is **per repository and machine-local**; a second brain is one body of knowledge that follows you across every project and syncs in git. And building the index yourself is what teaches you why retrieval works — which is what lets you fix the built-in one on the day it starts pulling the wrong note.

The difference from Part 2

Project memory answers *“how does this repo work?”* A second brain answers *“what have I already figured out, and what did I decide last time?”*

Part 2 makes the agent remember a project. This makes it remember **you** — across every repo, every machine, every session.

Retrieval, not volume

The instinct is to write a lot down. That is the wrong instinct.

A thousand notes the agent cannot find is worse than thirty it can. Once notes start resembling each other, retrieval degrades, and an agent that pulls the wrong note is worse than one that pulls none — it answers confidently from the wrong premise.

So the design goal is not capture. It is retrieval.

One fact per file

```

---
name: why-we-dropped-redis
description: Redis was removed from the pipeline in Aug 2026 because the cache
hit
    rate never went above 12%.
type: decision
---

Measured over three weeks: 12% hit rate, and the invalidation logic caused two
outages. Replaced with an in-process LRU.

**Why it matters:** if someone proposes Redis again, this is the measurement
that
says no. The hit rate is the number to re-check, not the outages.

Related: [[pipeline-architecture]]

```

Four types cover almost everything: `decision`, `preference`, `reference`, `gotcha`.

The `description` is not a summary — it is the **relevance test**, because it is often all the agent reads before deciding whether to open the file. “Redis notes” fails. The line above passes.

The index is the brain

One `MEMORY.md` at the root. One line per note. Never content, never a second copy.

```

- [Why we dropped Redis](why-we-dropped-redis.md) – 12% hit rate, not worth the
invalidation bugs
- [Prefers tables over prose](prefers-tables.md) – for anything comparative
- [Windows path gotcha in ffmpeg](ffmpeg-windows-paths.md) – backslashes break
the filter graph

```

The agent loads the index every session — one line each, cheap — and opens only what matters. That is the entire retrieval mechanism, and it works because the hooks are written to be **scannable**, not complete.

Delete the index and keep every note, and retrieval collapses almost completely. The notes are storage; the index is the brain.

Let the agent write it

In `~/ .claude/CLAUDE.md`:

```
## Second brain
Notes live in `~/brain/`. `MEMORY.md` there is the index; read it when a
question
might have been answered before.

Write a new note when I decide something non-obvious, state a preference about
how
I want things done, or hit a gotcha that cost more than ten minutes. One fact
per
file, add one line to MEMORY.md. Check for an existing note first and update
that
instead of duplicating. Never record what the code or git history already says.
```

It will over-record at first. Tighten the trigger and delete the junk — a brain that never prunes becomes noise, and noise is exactly what breaks retrieval.

Testing it honestly

Ask a question one of your notes answers, **phrased differently from the note**, in a fresh session, in an unrelated project. Testing with the note’s own words proves nothing.

What happened	What to fix
Found and used it	Nothing. Re-test next week.
Did not look	The instruction is too weak, or the index is not being read
Looked, picked wrong	The <code>description</code> hooks are too similar to each other

Then run the opposite test: ask something that should match **nothing**, and confirm it says so rather than stretching an unrelated note to fit. A brain that always finds something cannot be trusted.

Keep it plain

Markdown files in a folder. That is deliberate. It syncs with git, works with any agent, survives your tool of choice being replaced, and is readable by a human in five years.

One caution: this gets read by an agent and may be sent to a model provider. Nothing should be in it that you would not paste into a chat window.

Part 4 — Making it hold

Two mechanisms turn the above from “things you know” into “things that happen”.

Skills — for what you repeat

A skill is a folder with a `SKILL.md` in it. The frontmatter says when to use it; the body says how. The agent loads the body **only when the situation matches**, so a skill costs nothing until it is needed — which is exactly why a long workflow belongs in a skill rather than in your memory file.

<code>~/.claude/skills/<name>/SKILL.md</code>	(Claude Code, all projects)
<code>.claude/skills/<name>/SKILL.md</code>	(one project)
<code>~/.codex/skills/<name>/</code>	(Codex)

Claude Code watches those directories, so a new skill is picked up in the current session without a restart.

The `description` is the entire trigger. If a skill never fires on its own, the description is the thing to fix, and the test is simple: **if you only read that line, would you know whether to open the skill?**

The skill that ships with this guide is itself an example — a folder, a markdown file, and some reference files it loads on demand.

Hooks — for what must not happen

A markdown file asks. A hook decides.

Everything above is instruction: the agent reads it and usually complies. Usually is fine for style. Usually is not fine for “never push to main”.

A hook is a program the tool runs at a fixed point in its loop. It receives the proposed action as JSON on stdin, and **its exit code decides whether the action happens**. The model is not consulted.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash|PowerShell",
        "hooks": [
          {
            "type": "command",
            "command": "python ${CLAUDE_PROJECT_DIR}/.claude/hooks/guard.py"
          }
        ]
      }
    ]
  }
}
```

```
import json
import sys

payload = json.load(sys.stdin)
command = payload.get("tool_input", {}).get("command", "")

if "git push" in command:
    print(json.dumps({
        "hookSpecificOutput": {
            "hookEventName": "PreToolUse",
            "permissionDecision": "deny",
            "permissionDecisionReason":
                "Pushing is a human decision. Ask, do not push.",
        }
    })))
    sys.exit(0)

sys.exit(0)
```

There are two supported routes and you pick one per hook: **exit 0 and print JSON**, as above, or **exit 2 with the reason on stderr** — on a `PreToolUse`, your stderr text *is* the denial reason the agent is given. What matters is not which route you take but **whether you wrote a reason at all**.

Exit codes are not what a Unix habit expects: **0** means no decision, **2** blocks regardless of what you printed, and **1** — without valid JSON on stdout — is a *non-blocking* error, so the action proceeds anyway. A guard that crashes, or that returns 1 to mean “no”, lets the command straight through.

The other thing people get wrong is writing the reason for a log rather than for the agent. Nothing at all makes it try variations blindly. “Blocked.” makes it try a variation, get blocked, and loop. “Pushing is a human decision. Ask, do not push.” makes it stop and ask.

Two details decide whether a guard holds at all: match `Bash|PowerShell`, not `Bash` alone, because PowerShell is a separate tool and a Windows command sails straight past a Bash-only matcher; and write the script path as

`${CLAUDE_PROJECT_DIR}/.claude/hooks/guard.py`, because a relative path resolves against wherever Claude happens to be when the hook fires.

Which rules deserve a hook? One test:

Would you be genuinely upset if this happened once, at 2am, in a long session where you were not reading carefully?

Yes → hook. No → leave it in the markdown. Naming conventions and comment style do not need enforcement; they need suggestion. Force-pushing, deleting branches, touching `.env`, running migrations — those are hooks.

Three good hooks beat twenty. Every hook is something that can misfire at the worst possible moment, so make each one fail open on its own errors.

Where to start

Do not do all of this at once. In order of return:

1. **Write the memory file.** Twenty minutes, and it pays back the same week. Nothing else works properly without it.
2. **Run one fan-out** across three dimensions of a real problem. It will show you immediately whether your file is good enough.
3. **Add one hook** for the one thing you would hate to have happen.
4. **Start the brain** when you next catch yourself solving something for the second time. Not before — an empty structure will stay empty.
5. **Write a skill** for whatever you have now explained three times.

Each of those is independent and each is useful alone. The compounding starts at about step three, when the fan-out in step two starts inheriting the memory from step one.

Do it instead of reading it

Reading this once will not change how you work. Building it will.

The repository this guide came from ships a **skill that teaches these tracks interactively** — it asks what you want to learn, then walks you through building it, with a real project at the end of each track and checks that catch the specific mistakes people make.

```
github.com/Life696969/coding-agents-guide
```

Install it, run it, and pick a track.

— *Mudit Jain, @ai_with_mudit*